



```
irb(main):103* class Animal
irb(main):104*   private...
```

Your observation is correct based on older versions of Ruby, but Ruby's behavior regarding `self` and private methods changed in **Ruby 2.7**.

Prior to Ruby 2.7, your code would have raised a `NoMethodError`. In modern Ruby, `self.make_sound` is perfectly valid.

The Private Method Rule Change

- Old Rule (Pre-2.7):** Private methods could never be called with an explicit receiver, even if that receiver was `self`.
- New Rule (Ruby 2.7+):** Private methods can be called with `self` as an explicit receiver. They still cannot be called with any other receiver (like

Ask anything



To call a setter method, you are forced to use an explicit receiver: `self.weight = 50`. Before Ruby 2.7, this meant setter methods could never be truly private because they required `self..`

Allowing `self` with private methods fixed this design inconsistency.

What is Still Forbidden?

Private methods are still protected from external access. You still cannot call them on a specific instance variable from the outside.

ruby

```
dog = Dog.new
```

```
# This will fail with a NoMethodError
dog.make_sound
```

Use code with caution.



Would you like to explore how **protected** methods differ from private methods in Ruby, or see how to use **metaprogramming** to bypass these visibility rules entirely?



```
In Ruby, if you write weight = 50 inside a class, Ruby treats
weight as a local variable. To call a setter method, you are ...
```

Here is how the restriction used to break private setter methods before Ruby 2.7, and how it works today.

The Code Example

ruby

```
class Person
  def initialize(age)
    # Pre-Ruby 2.7: This raises NoMethodError
    # Modern Ruby: This works perfectly
    self.age = age
  end

  private

  def age=(new_age)
    @age = new_age
    puts "Age securely updated to #{@age}"
  end
end
```

```
Person.new(25)
```

Use code with caution.



The Pre-Ruby 2.7 Dilemma

Before Ruby 2.7, the code above failed. Developers faced a catch-22 trying to use a private setter:

- Attempt 1 (`self.age = age`):** Crashed because Ruby banned using `self.` with private methods.
- Attempt 2 (`age = age`):** Failed silently because Ruby interpreted this as creating a new local variable named `age`, never calling the setter method.

The Workarounds (Old vs New)

- Before Ruby 2.7:** Developers had to bypass the setter completely and assign directly to the instance variable (`@age = age`), or use metaprogramming (`send(:age=, age)`).
- Ruby 2.7 and later:** You simply type `self.age = age`. It safely routes to your private setter while keeping it hidden from the outside world.

Would you like to see how to bypass privacy using `send`, or should we look at how **protected setters** behave differently?



3 sites

Class Inheritance

As of Ruby 2.7, it is now legal to call private methods with a literal self as the caller. No...

Launch School

Class Inheritance

As of Ruby 2.7, it is now legal to call private methods with a literal self as the caller. No...

Launch School

Classes and Objects II

Use self when calling setter methods from within the class. In our earlier example we showed that self was necessary i...

Launch School